Understanding Wavetable Synthesis

Let's understand the concept of wavetable synthesis by designing a sine wave generator.

Here is the general formula:

$$s(t) = Asin(2\pi ft + \phi)$$

Where:

- A: **Amplitude**, the peak deviation of the function from zero.
- f: **frequency**, the number of oscillations (cycles) that occur each second of time.
- ϕ: **phase**, specifies (in radians) where in its cycle the oscillation is at t = 0.

Having this formula, we are able to compute the value of a sine function at any point in time t.

Remember that in digital audio, time passes in **sample frames**

Samples are related to time by a **sampling rate** (how many discrete values are produced per second).

We can compute time in seconds with this formula:

$$t = \frac{n}{f_s}$$

Where:

- n: **sample index**, an integer, representing where in the sequence a sample occurs. It could be the current index since the running of the program.
- f_s: **sampling rate**, the number of samples taken per second. Where $f_s = \frac{1}{T_s}$. T_s is the **sampling period** (the time interval). For example, with a sampling period of 0.25 of a second, we can calculate the sampling rate by taking the inverse (1/0.25). This will give us a value of 4 as the sampling rate.

With a sampling rate of 44.1k, we can find t by trying different values of n.

$$n = 5$$

$$5/44100 = 0.00011338 \text{ seconds}$$

$$n = 22050$$

$$22050/44100 = 0.5 \text{ seconds}$$

$$n = 114660$$

$$114660/44100 = 2.598 \text{ seconds}$$

For a simple wavetable synth program, we can ignore the phase since it will always start at 0.

Here is the formula rewritten:

$$s[n] = Asin(2\pi fn/f_s)$$

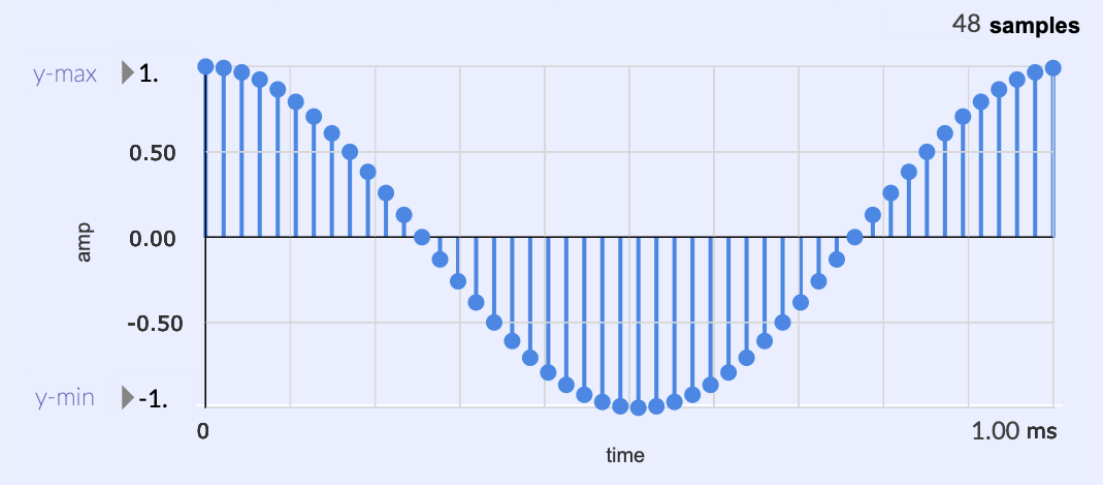
A rabbit hole topic is looking into the question of how computers calculate the value of `sin()` or `cos()`.

One common method is using **Taylor Expansion**. But my point is that if we are using a sample rate of 44.1k, that means we would be doing this calculation 44,100 times per second, which can be computationally expensive. Modern computers can handle it, but it is a waste of resources.

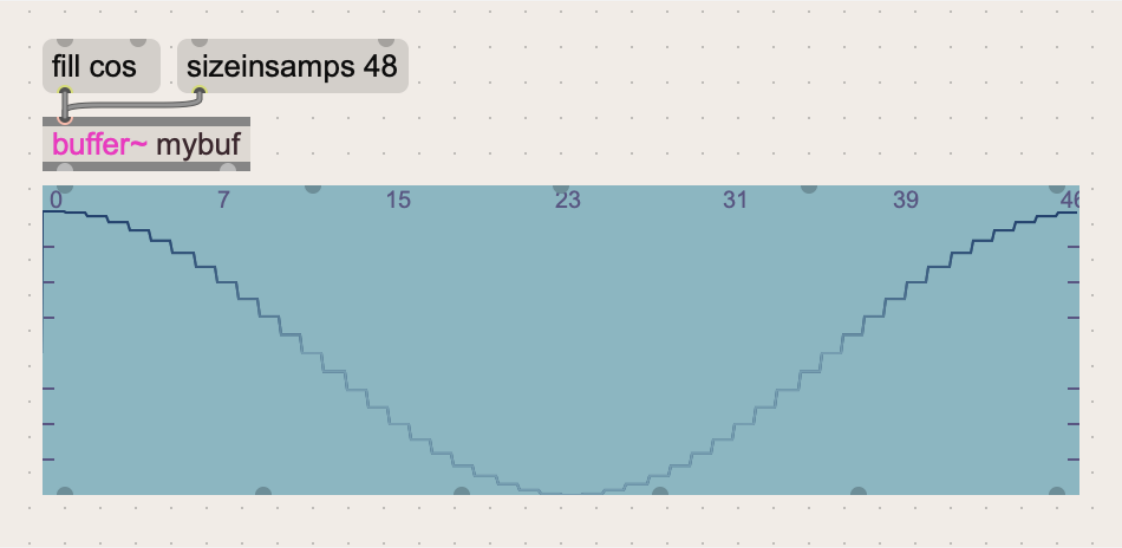
Instead, we could just calculate one cycle, store the values in a table, and then use that table to lookup/approximate values for any frequency wave.

Wavetable Definition

A wavetable is an array in memory that stores a fragment of a waveform



A plot of the cosine wave values from a wavetable.



In Max/MSP, we use a buffer object, in this case it is 48 samples long, and a cosine function is applied.

- 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
- 1. 0.991445 0.965926 0.92388 0.866025 0.793353 0.707107 0.608761 0.5 0.382683 0.258819 0.130526 0. -0.130526 -0.258819 -0.382683 -0.5 -0.608761 -0.707107 -0.793353 -0.866025 -0.92388 -0.965926 -0.991445 -1. -0.991445 -0.965926 -0.92388 -0.866025 -0.793353 -0.707107 -0.608761 -0.5 -0.382683 -0.258819 -0.130526 -0. 0.130526 0.258819 0.382683 0.5 0.608761 0.707107 0.793353 0.866025 0.92388 0.965926 0.991445

The data in the wavetable. The floating-point numbers are the amplitude levels that are stored.

Phase vs. Table Index Ratio

Remember that the period of a sine wave form is exactly 2π long.

If we store one period of a sine wave in a table, we can compute the index that corresponds to a specific phase, by using the ratio below.

The ratio of the phase θ relative to the period is exactly the same as the table index relative to the table size:

$$\frac{k}{L} = \frac{\theta}{2\pi}$$

Where:

- L: **Length of Wavetable**
- k: **wavetable index** (must be between 0 - L)
- θ: **phase in radians** (must be between 0 - 2π)

We can find the value of k:

$$k = \frac{\theta L}{2\pi}$$

If we have a wavetable that is 64 samples long, and we want to know the index k of a wave with a phase of 3.14, then:

- L = 64
- θ = 3.14

$$k = \frac{3.14 * 64}{2\pi}$$

$$k = \frac{200.96}{2\pi}$$

$$k = 32$$

In this case we got a result that is exactly an integer, but in most cases k will be between two sample points. For example, with a phase value of 3.0, we get a k value of 30.573.

There are several strategies we can use.

We could use **truncation**, which is just removing the decimal information. A value of 30.573 would just turn to 30. This is the same as using the `floor()` function.

We could also **round** k to the nearest integer. 30.573 would turn to 31.

We could use **linear interpolation**, which involves the weighted sum of the two values in the wavetable that are nearest.

Using `floor()` (0th Order Interpolation)

The easiest implementation is to use the `floor()` function for truncating the k value to an integer.

In order to hear our sine tone, we have to step through the wavetable a certain amount at the sampling rate to hear our desired frequency tone.

We can calculate the phase increment, which is the amount of radians to increment each sample frame.

The formula for that is:

$$\Delta = \frac{f}{F_s} * L$$

$$\Delta = \frac{440}{44100} * 1000 \approx 9.977$$

Or we can use the **index increment**, which can be found using this formula:

$$\frac{fL}{f_s}$$

Where:

- f: **the desired frequency in Hz.** If you want to play an A4 note, the f value would be 440.

In the Max/MSP tutorials, they talk about how the `cycle~` MSP object is a 512 (maybe 16384 now) sample wavetable. They use the formula where I is the index increment, f is the desired frequency, L is the sample length of the wavetable, and R is the audio sampling rate.

`cycle~` uses linear interpolation.

$$I = \frac{fL}{R}$$

Where:

- R is f_s, the sampling rate
- I is index increment

Other MSP objects use more computationally expensive polynomial interpolation.

Here is a python implementation of a wavetable oscillator program using the `floor()` method:

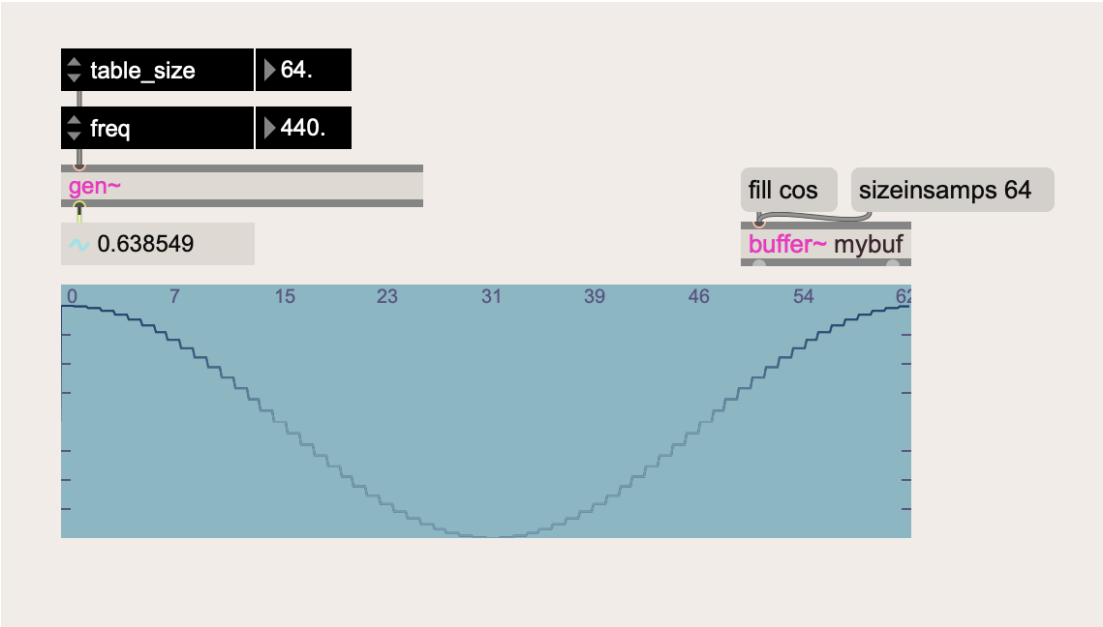
```
import numpy as np
import scipy.io.wavfile as wav


def main():
    sample_rate = 44100
    f = 440
    t = 3

    wavetable_length = 64
    wave_table = np.zeros((wavetable_length,))
    output = np.zeros((t * sample_rate,))

    # iterate through wavetable to generate values based on n
```

```
    for n in range(wavetable_length):
        wave_table[n] = np.sin(2 * np.pi * n / wavetable_length)

    index = 0

    ## calculate the index increment
    index_increment = f * wavetable_length / sample_rate

    # iterate through .wav file (44.1k samples * 3)
    for n in range(output.shape[0]):
        # the wave sample value written in output array
        output[n] = wave_table[int(np.floor(index))]
        # increment index
        index += index_increment
        # keep index within wavetable size
        index %= wavetable_length

    # creates a 3 second .wav file of a sine tone
    wav.write("sine440Hz.wav", sample_rate, output.astype(np.float32))


if __name__ == "__main__":
    main()
```

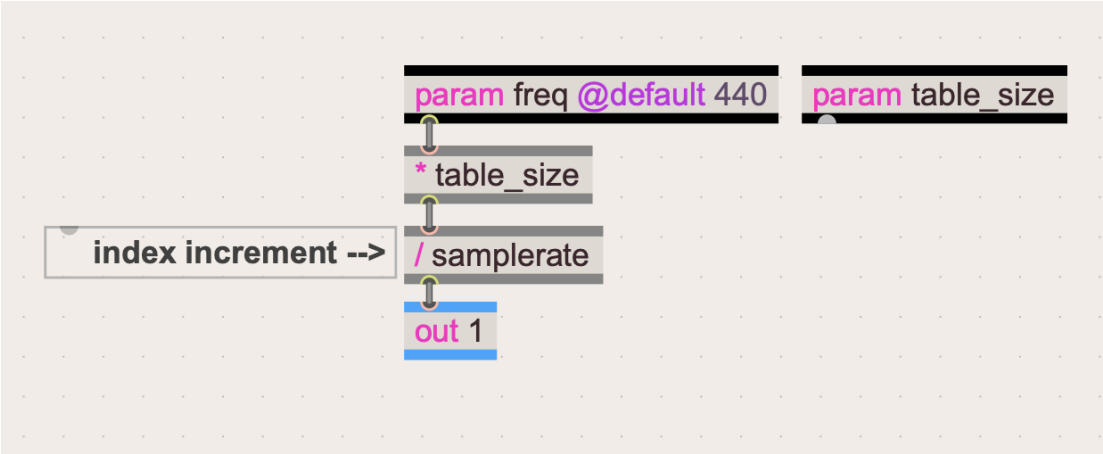
When printing out the value of the index increment, the program returns:

```
0.6385487528344671
```

To recreate this same program in Max/MSP gen~, let's first confirm we get the same value.

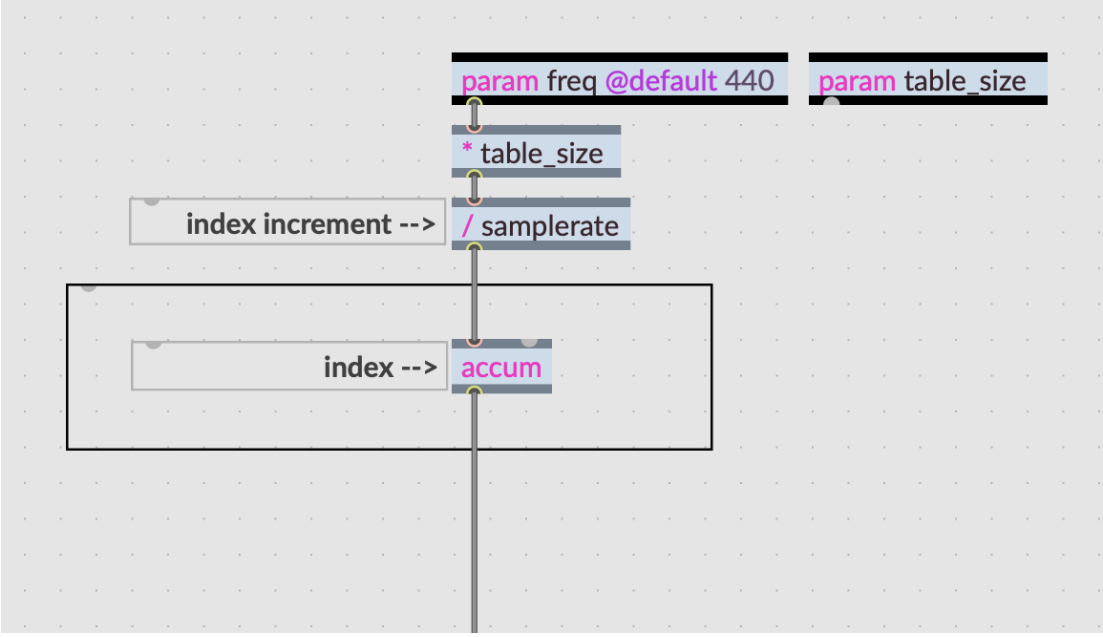


We are using the same parameters. We have a buffer/wavetable that is 64 samples long, filled with a cosine function, and we want a tone with a frequency of 440 Hz.



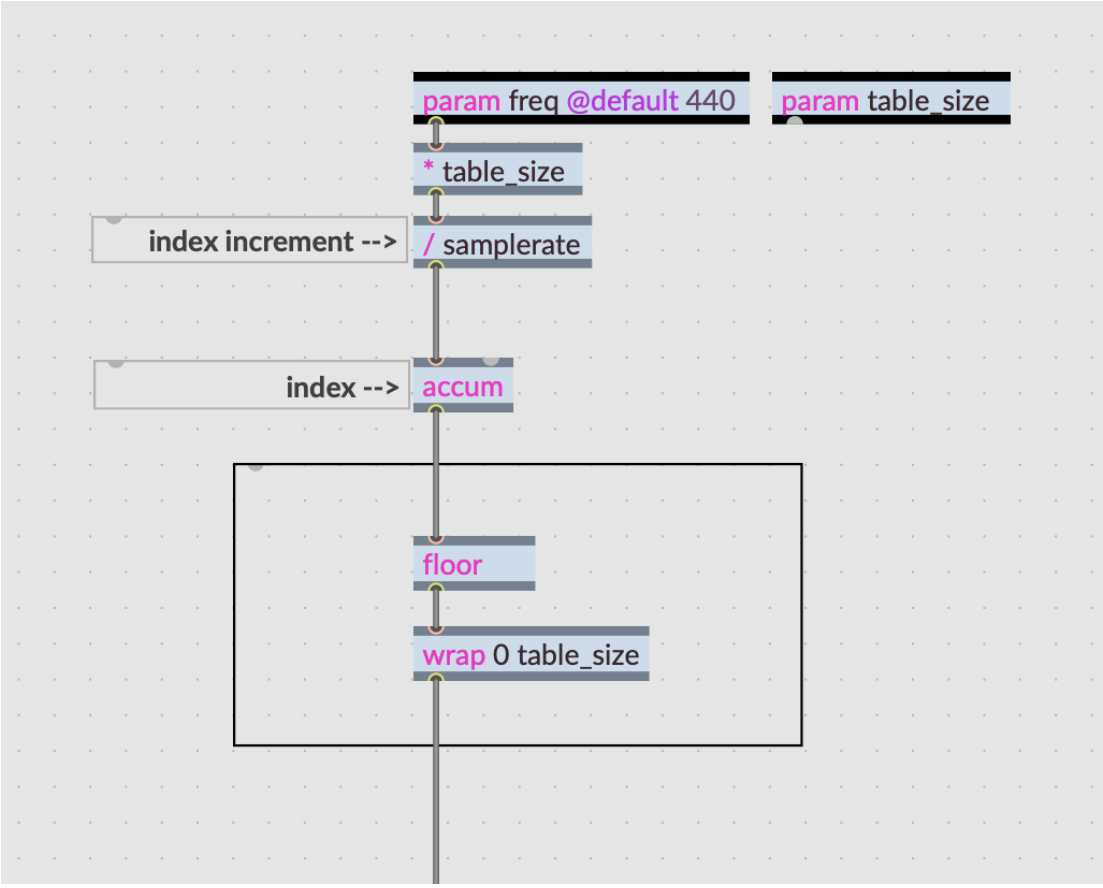
We are following the same equation where we multiply table size and frequency, then divide that result with the sampling rate.

The next step would be to accumulate that index increment to hold the value k.



The 'accum' operator continues to increment by the index increment. This will be our k

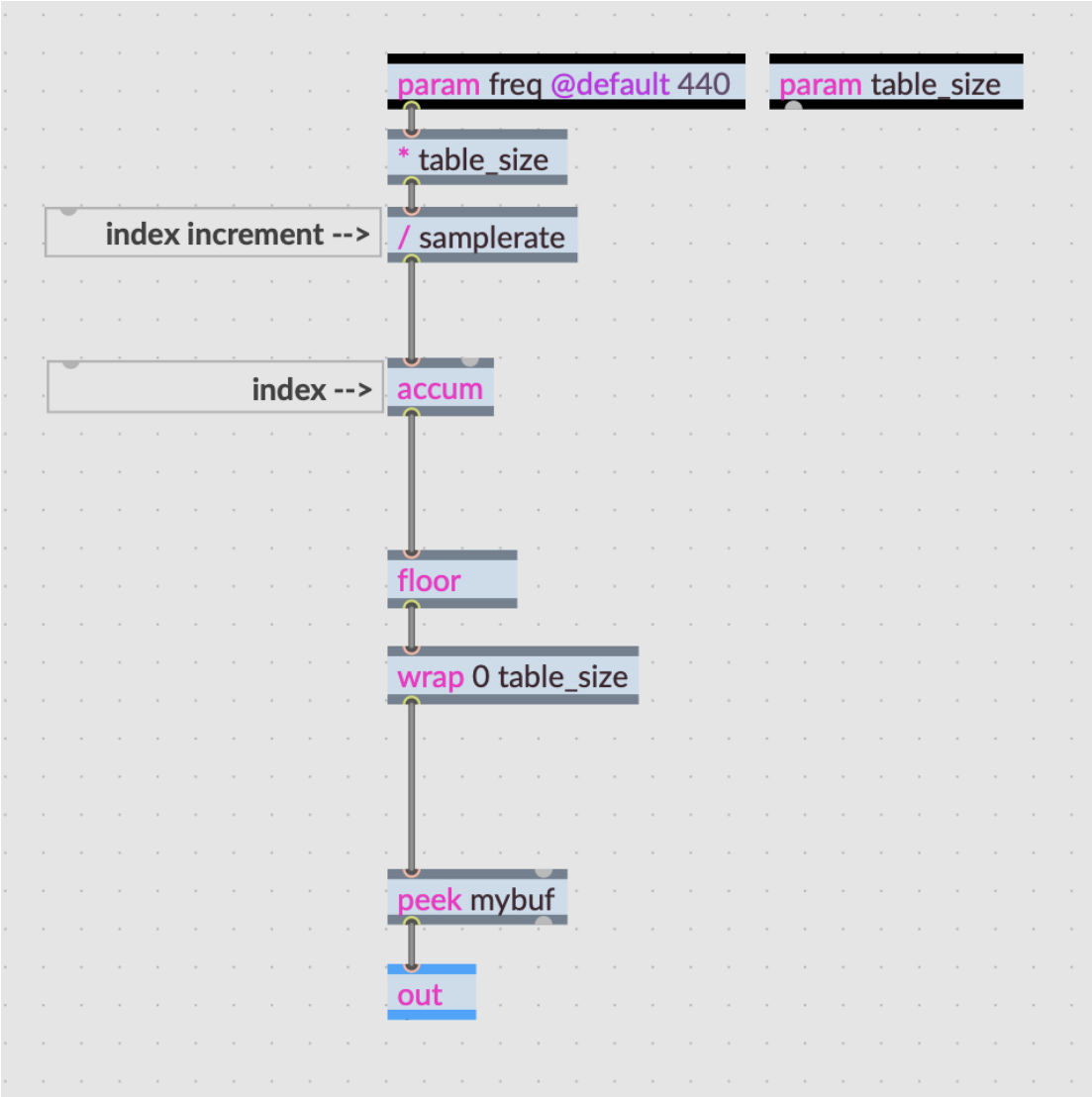
Now we can go ahead and do a table lookup. We can use the peek operator, but first we must round it down to convert the value into an integer, and we must keep it within the 0-63 range.



outlet value of the 'wrap' operator will be the wavetable index.

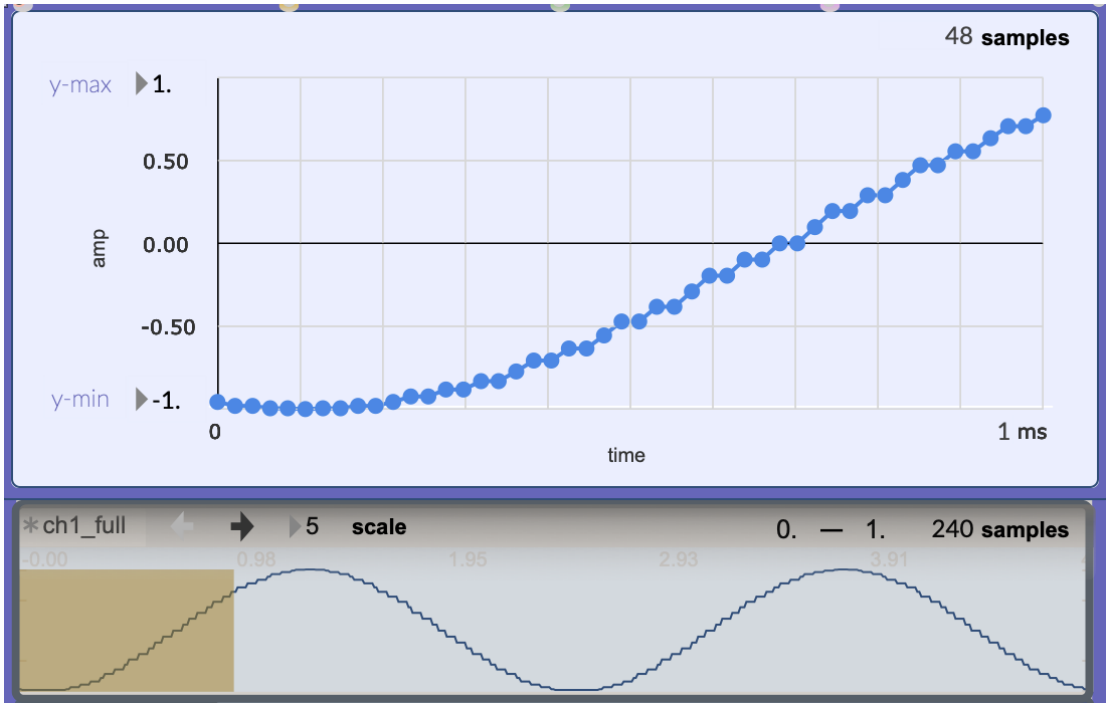
This truncation method with the `floor` operator is **0th-order interpolation.**

Here is the final gen~ patch:

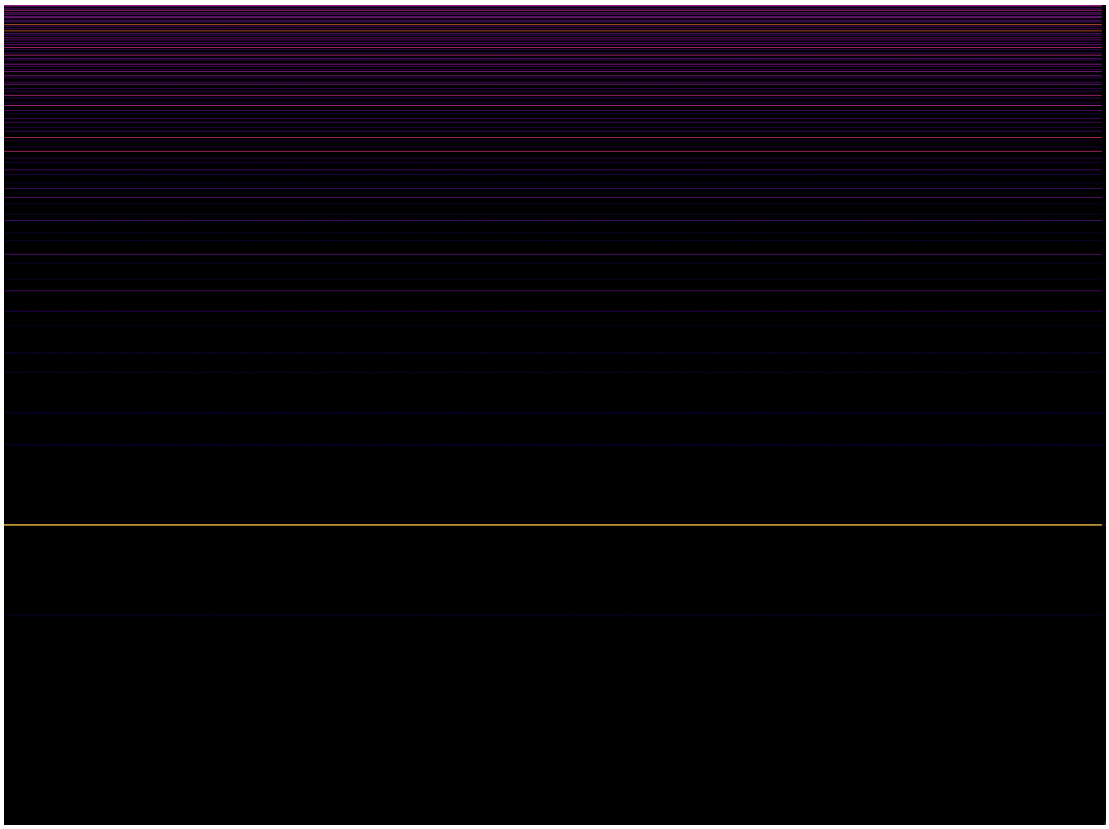


We are using the 'peek' operator to lookup the value corresponding to the wavetable index.

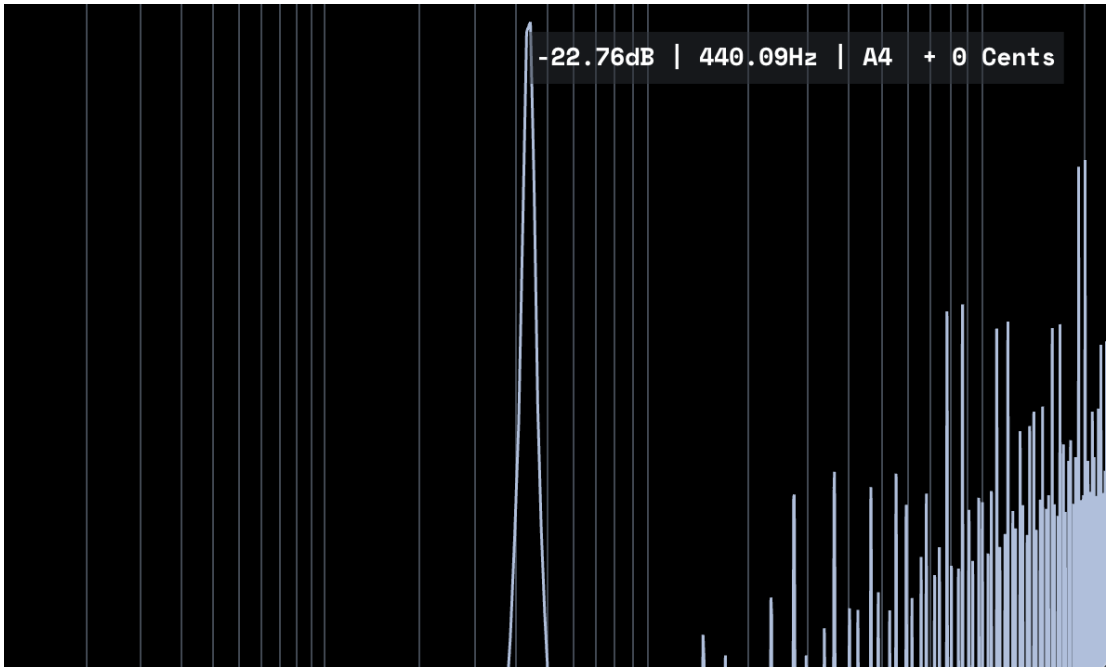
Here is a 1 ms plot of the waveform:



You can see that the sinusoid is recognizable, but choppy.



The spectrogram shows the spectral consequence of the choppy waveform. It results in high-frequency partials that cause an annoying ringing.



The 440 Hz tone is present. Along with a lot of aliasing.

Linear Interpolation

Let's do a linear interpolation implementation of both the python and gen~ programs. This is also known as **1st-order interpolation.**

Here is `lerp()` function in python:

```
def lerp(wave_table, index):
    truncated_index = int(np.floor(index))
    next_index = (truncated_index + 1) % wave_table.shape[0]

    next_index_weight = index - truncated_index
    truncated_index_weight = 1 - next_index_weight

    return (
        truncated_index_weight * wave_table[truncated_index]
        + next_index_weight * wave_table[next_index]
    )
```

Linear interpolation is simply estimating a value between two known points, based on how close/far we are from those two points. Visually, it is like drawing a line between the two points and finding the point in that line.

This is essentially calculating a **weighted average.**

You can find the wanted value by using the formula:

$$y = y_1 + (x - x_1) * \frac{(y_2 - y_1)}{(x_2 - x_1)}$$

First identify the two points (x_1, y_1) and (x_2, y_2):

Our `truncated_index` and `next_index` will be our x_1 and x_2.

The corresponding values returned by our wavetable lookup will be our y_1, and y_2.

As you can see, it does the same `floor` function to get the nearest sample that it just passed. The next sample is simply this result + 1. It also uses the modulo operator to keep it within the bounds of the wavetable length.

```
output[n] = lerp(wave_table, index)
```